

실시간 모니터링과 제어를 지원하는 IoT 기반 스마트 환경관리 시스템 SEMS – Smart Environment Management System

김현호, 나은주, 임종원, 한민서

Abstract 본 연구에서는 실내 공기질과 에너지 효율을 동시에 관리하기 위한 IoT(Internet of Things) 기반 스마트 환경관리 시스템(SEMS, Smart Environment Management System)을 구현하였다. 아두이노와 라즈베리파이를 통해 미세먼지, 온습도, 전류 데이터를 수집하고, MQTT(Messaging Queuing Telemetry Transport) 프로토콜로 전송하여 웹 UI에서 실시간 시각화와 환기 장치 제어를 지원한다. 이를 통해 사용자는 공기질 개선과 에너지 소비를 직관적으로 확인하며 능동적으로 환경을 관리할 수 있다.

Keywords: Arduino, Raspberry Pi, IoT, MQTT

1. 서론

최근 IoT 기술은 센서 네트워크와 클라우드 플랫폼 간 실시간 데이터 교환을 가능하게 하며 스마트홈, 스마트빌딩, 공공시설 등에서 환경 모니터링과 제어 시스템 구축에 활발히 활용되고 있다. 에너지 경제연구에 따르면 여름철 공공기관 적정실내온도 준수 규정의 전기 소비 절감 효과를 시간별 마이크로데이터에 온도 반응 함수와 이중차분법을 적용하여 인과적으로 분석하였다[1]. 그 결과, 전력 수요가 집중되는 오후 시간대, 그리고 집중의 정도가 높아지는 외부 온도가 높은 날에 전력 소비가 두드러지게 나타나는 한계가 있었다. 이러한 한계는 실내 환경 관리의 기술적 개선 필요성을 시사한다.

본 연구에서는 이러한 한계를 극복하기 위하여 MQTT 기반 웹페이지 구조를 적용한 스마트 환경관리 시스템(SEMS, Smart Environment Management System)을 설계하였다. 아두이노와 라즈베리파이를 통해 수집된 미세먼지, 온습도, 전류 데이터는 MQTT 브로커를 거쳐 HiveMQ Cloud로 실시간으로 전송되고, Spring Boot 서버와 React 웹 UI를 통해 즉시 시각화되며 환기 장치 제어(자동/수동 모드) 기능을 제공한다. 사용자는 별도의 전용 프로그램 설치 없이 웹페이지를 통해 어디서든 실시간 데이터를 확인하고 직접 제어할 수 있으며, 이는 단순히 데이터를 관찰하는 수준을 넘어 상호작용적인 환경 관리 경험을 가능하게 한다.

본 연구의 주요 기여점은 세 가지로 요약된다. 첫째, MQTT 기반의 개방형 웹 아키텍처를 적용하여 다양한 IoT 장치와의 연동 및 확장이 용이한 구조를 제시하였다. 둘째, 센서 데이터뿐 아니라 전류 데이터를 함께 분석하여 환경 관리와 에너지 효율성을 동시에 고려할 수 있는 기능을 구현하였다. 셋째, 관리자와 일반 사용자의 권한을 분리하고, 실시간 데이터 시각화 및 제어 기능을 웹페이지에 통합함으로써 상호작용성과 실용성을 강화하였다. 이를 통해 본 시스템은 실시간 환경 관리와 자동화된 제어 시스템을 효과적으로 제공할 수 있을 것으로 기대한다.

2. 주요기술

2.1 아두이노(Arduino)

아두이노는 GPIO 핀에 연결한 센서와 액추에이터를 직접 제어할 수 있는 오픈소스 마이크로컨트롤러 보드이다. 라즈베리파이는 아두이노에서 수집된 데이터를 저전력 환경에서도 안정적으로 동작할 수 있는 MQTT 프로토콜을 활용하여 클라우드 데이터베이스로 전송하여 관리한다. 본 프로젝트에서는 [그림 1]과 같은 온도·습도 센서와 미세먼지 센서, 전류 센서로 데이터를 수집하고, 수집한 데이터를 분석하여 모터를 제어하는 역할을 담당하였다.

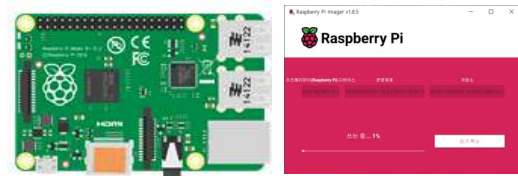


[그림 1] 입출력 장치

2.2 라즈베리파이(Raspberry Pi)

[그림 2]의 라즈베리파이는 초소형 컴퓨터로, 본 프로젝트에서 아두이노와 MQTT 클라우드 서버(HiveMQ) 사이의 게이트웨이 역할을 수행하였다. 라즈베리파이는 센서 데이터를 받아 네트워크 환경에 맞게 변환·전송하며, 또한 Flask 기반 Python 모듈을 실행하여 AI 모드 기반으로 구동하는 핵심 노드로도 사용되었다.

본 연구에서는 [그림 2]의 Raspberry Pi OS가 탑재되었다. 이 운영체제 환경 위에서 Python 기반의 MQTT 클라이언트 코드가 실행되며, 아두이노로부터 수신한 센서 데이터를 HiveMQ 클라우드로 전송하는 핵심적인 역할을 수행한다.

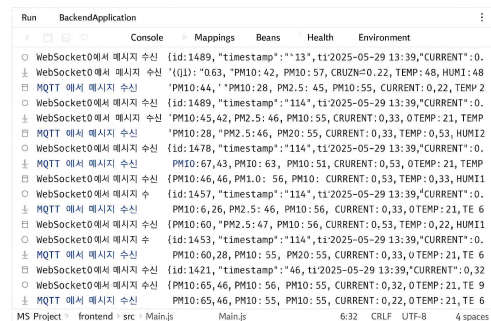


[그림 2] Raspberry Pi

2.3 MQTT(Message Queuing Telemetry Transport)

메시지를 발행하고 구독하는 방식으로 동작하는 매우 가벼운 통신 프로토콜이다. 주로 사물 인터넷(IoT) 기기처럼 저전력, 저사양 환경에서 효율적으로 데이터를 주고받기 위해 사용된다. 동작 방식은 다음과 같다. 발행자(Publisher)가 특정 토픽(Topic)을 정해 브로커(Broker)에게 데이터를 보낸다. 구독자(Subscriber)는 자신이 관심 있는 토픽을 브로커에게 미리 알려두고 구독한다. 브로커는 발행자에게서 받은 데이터를 해당 토픽을 구독하는 모든 구독자에게 전달해준다.

본 연구에서는 데이터 파이프라인 구축 과정에서 MQTT를 적용하였다. [그림 3]처럼 [그림 1]의 센서에서 데이터를 수집하고 라즈베리파이를 통해 HiveMQ 클라우드로 MQTT 프로토콜을 사용하여 데이터를 발행한다. 이후, Spring 서버는 HiveMQ에 구독하여 센서 데이터를 수신하고 처리한다.

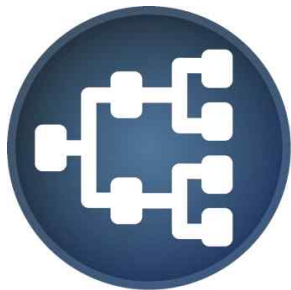


[그림 3] MQTT를 활용한 데이터 전송

2.4 HiveMQ Cloud

HiveMQ Cloud는 클라우드 기반의 완전 관리형 MQTT 브로커 서비스다. 사물 인터넷(IoT) 기기들이 서로 메시지를 빠르고 안정적으로 주고받을 수 있도록 중개해주는 역할을 하고 복잡한 서버를 직접 설치하고 관리할 필요 없이, 바로 MQTT 메시징 시스템을 구축하고 사용할 수 있게 해준다.

본 연구에서는 MQTT Explorer를 사용했는데 데스크톱 클라이언트 도구로서, HiveMQ 클라우드와 같은 MQTT 브로커에 접속하여 토픽에 발행된 데이터를 실시간으로 구독(Subscribe) 및 모니터링하고, 새로운 메시지를 임시로 발행(Publish)하여 시스템의 데이터 흐름과 통신 상태를 테스트 및 검증하는 용도로 활용된다.



[그림 4] MQTT Explorer

2.5 WebSocket

서버와 클라이언트(웹 브라우저) 간에 실시간으로 양방향 통신을 가능하게 하는 기술이다. 한 번 연결이 맺어지면 계속 그 통로를 열어두고, 양쪽에서 원할 때마다 자유롭게 데이터를 주고받을 수 있다. 기존의 HTTP 통신은 클라이언트가 요청을 보내야만 서버가 응답할 수 있는 단방향성 구조다. 실시간처럼 보이게 하려면 클라이언트가 주기적으로 물어봐야 하는 비효율적인 방식(폴링, Polling)을 사용해야 했다. 반면, 웹소켓은 최초 연결(Handshake) 이후에는 연결을 계속 유지한다. 본 연구에서는 DB에 저장된 데이터를 웹소켓 연결을 통해 그래프 분석에 사용하거나 유저와 관리자 간 실시간 채팅에 사용된다.

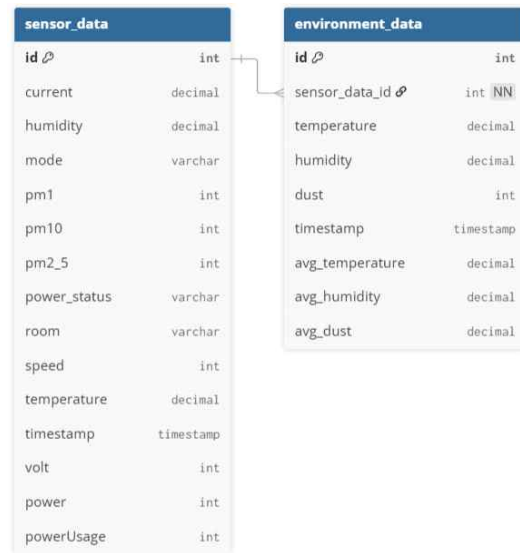
3. 데이터베이스 및 분석

3.1 MySQL

MySQL은 표준 데이터베이스 질의 언어인 구조화 질의 언어(SQL: Structured Query Language)를 사용하는 공개 소스의 관계형 데이터베이스 관리 시스템(RDBMS)이다. 기본적인 백엔드 언어인 JAVA와 C, C++, PHP, 파이썬 스크립트 등의 응용프로그램과 상호 연동이 잘되면서 오픈소스로 개발되는 무료 프로그램이다. 본 논문에서는 데이터베이스 관리를 위해 MySQL을 사용한다.

3.2 테이블 설계

‘스마트 환경관리 시스템’을 구현하기 위해 4개 테이블을 설계하였다. 우선, 환경 데이터 측정 값을 관리하기 위해 sensor_data 테이블과 nvironment_data 테이블을 둔다.



[그림 5] 센서 데이터 관리

먼저, 아두이노 센서를 통해 온도, 습도, 미세먼지 데이터를 초 단위로 측정하여 sensor_data 테이블에 저장한다. 초 단위로 저장된 온도, 습도, 미세먼지 데이터는 다시 분 단위로 평균값을 계산하여 environment_data 테이블에 저장한다. 사용자는 environment_data 테이블을 토대로 실

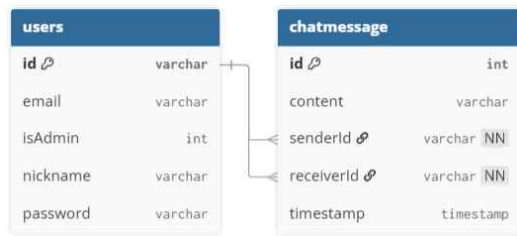
내 환경 데이터 추이를 확인할 수 있다.

[그림 5]는 sensor_data 테이블과 environment_data 테이블의 관계도(ERD: Entity Relationship Diagram)를 보여주고 있다.

또한 사용자 관리, 사용자와 관리자 간에 실시간으로 주고받는 메시지 관리를 위해 users와 chatmessage 테이블을 둔다.

users 테이블은 로그인에 사용할 id와 email, 비밀번호, 관리자 여부(isAdmin) 등을 저장하며, chatmessage 테이블은 사용자 ID(senderId)와 관리자 ID(senderId(receiverId))를 통해 사용자와 관리자가 주고 받는 채팅 메시지를 저장하여 관리 한다.

[그림 6]은 users 테이블과 chatmessage 테이블의 관계도(ERD: Entity Relationship Diagram)를 보여주고 있다.

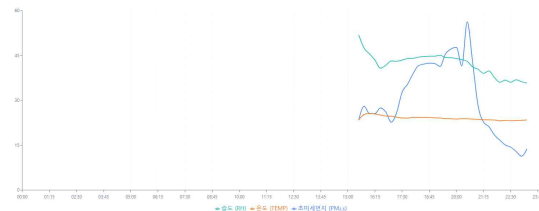


[그림 6] 사용자 관리

3.3 센서 데이터 분석

관리자는 접속한 ‘스마트 환경관리 시스템’의 웹 화면을 통해 아두이노 실내 센서가 수집한 환경 데이터를 실시간으로 확인하고 기기를 제어할 수 있다.

또한 SEMS는 [그림 5]와 같이 환경 센서 측정 값을 분 단위로 가공하여 관리하는 environment_data 테이블을 분석하여 [그림 7]의 실내 환경 데이터 추이 그래프를 제공한다. 관리자는 일정 시간 동안 측정하고 가공한 온도·습도·미세먼지 값의 변화를 그래프를 통해 실시간으로 파악할 수 있다. 관리자는 이 그래프를 통해 환경 변수 제어를 위한 의사 결정 정보로 활용한다.



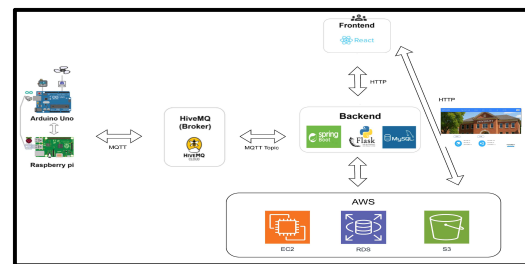
[그림 7] 데이터 분석 표

[그림 7]은 실제 구현 시스템으로 수집한 환경 데이터를 시간 변화에 따라 몇 시간 동안의 온도, 습도, 미세먼지의 변화를 시각화하여 보여주고 있는 그래프이다.

따라서 관리자나 사용자는 로그인한 후, 실내 환경 데이터를 조회한 후, 공기청정기 모드, 풍량, 전원 상태 등 제어 명령을 내릴 수 있으며, [그림 7]의 그래프에서 확인되는 데이터 변화를 보면서 제어 효과를 실시간으로 확인할 수 있다, 결국 SEMS를 활용하여 주변 환경 변화를 실시간으로 반영하여 환경 데이터를 관리할 수 있고 에너지 효율을 높일 수 있다.

4. 구현

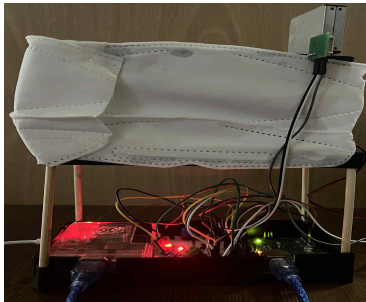
4.1 시스템 아키텍처 및 데이터 흐름



[그림 8] 시스템 동작 과정

본 시스템의 데이터 처리는 크게 모니터링과 제어의 두가지 흐름으로 동작한다. 첫째, 모니터링 흐름은 Arduino가 측정한 환경 데이터를 Raspberry Pi로 전송하면, RPI는 이를 MQTT 프로토콜을 통해 HiveMQ Cloud로 발행한다. Spring Boot 서버는 해당 토픽을 구독하여 데이터를 수신한 후 MySQL 데이터베이스에 저장하고, 가공된 데이터를 웹 클라이언트로 전달한다. 둘째, 제어흐름은 사용자가 웹 UI에서 환기 장치 제어 명령을

내리면, Spring Boot 서버가 제어 신호를 발행하고, 이를 구독하고 있던 Raspberry Pi와 Arduino가 순차적으로 팬의 전원 및 속도를 조절하는 방식으로 동작한다. [그림 9]는 센서들과 Arduino, Raspberry Pi로 구성된 하드웨어이다.



[그림 9] 환기청정기

4.2 프론트엔드

본 시스템의 웹 인터페이스는 사용자에게 직관적인 모니터링 및 제어 환경을 제공하기 위해 React 라이브러리를 기반으로 구현되었다. UI를 독립적인 컴포넌트(Component) 단위로 설계하여 개발의 효율성과 유지보수성을 높였으며, 이를 통해 실시간 센서 데이터, AI 예측 그래프, 그리고 환기 장치 제어 패널이 통합된 대시보드를 구축하였다. 특히, IoT 시스템의 특성상 다수의 센서 데이터가 빈번하게 갱신되는 환경임을 고려하여, React의 가상 돔(Virtual DOM) 기술과 최적화된 상태 관리(State Management) 기법을 적극 활용하였다. 이를 통해 데이터 변화에 따른 불필요한 렌더링을 최소화함으로써, 화면의 깜빡임이나 지연 없이 부드럽고 일관된 사용자 경험(UX)을 제공하도록 하였다.

4.3 백엔드

Spring Boot는 본 시스템의 중추적인 역할을 담당하는 서버 애플리케이션으로, 회원 관리, 데이터베이스 연동, 권한 제어 등 핵심 비즈니스 로직을 수행한다. 구체적으로 MQTT 브로커(HiveMQ)를 통해 수신한 센서 데이터를 MySQL 데이터베이스에 영구 저장

하고, 이를 가공하여 REST API 형태로 React 프론트엔드에 전달하는 데이터 파이프라인의 허브 역할을 한다. 또한, 시스템의 보안성과 실용성을 동시에 확보하기 위해 관리자와 일반 사용자의 권한을 엄격히 구분하였다. 이에 따라 환기 장치 제어 권한과 데이터 조회 범위를 차등 적용하여, 비인가자의 불필요한 기기 조작을 방지하고 효율적인 운영이 가능하도록 구현하였다.

5. 결론

날씨와 시간에 영향을 받는 비효율적인 전력 소비를 보완하고자 스마트 환경관리 시스템을 개발하였고, 데이터 시각화를 통해 공기질과 에너지 소비를 직관적으로 파악하고자 하였다. 뿐만 아니라 실시간 채팅과 환기 장치 제어를 통해 상호작용을 하여 사용자도 환경 개선에 기여할 수 있도록 하고 싶었다. AI 모드를 활용해 전력 소비량을 얼마나 절약할 수 있는지에 대한 지표는 데이터 분석의 과제로 남는다.

감사의 글

프로젝트에 앞서 저희 조를 잘 이끌어 주시고 많은 조언과 도움을 주신 정석용 교수님께 감사합니다.

참고문헌

[1] 조진만·김종우·박상규, “늘었지만 늘지 않다:적정실내온도 준수 규정의 정책효과 분석”, 에너지경제연구 제 24 권 제 2 호, pp. 1~38

			
김현호 (통신)	나은주 (UI)	임종원 (AWS)	한민서 (CRUD)